RTL TO GDS-II WORKSHOP

Project Report on

"RTL to GDS-II implementation of Floor($\log_2$(X)) for 8-bit Unsigned Inputs"

Submitted by

Mohammad Huda Ansari (23BEC147)

Department of Electronics and Communication Engineering,

School of Technology,
Pandit Deendayal Energy University (PDEU)

Submitted to

Dr. Arpan Desai & Dr. Ankur Changela

RTL2GDS-PDEU

Problem Statement

I. Implement the equation Y = floor(log2(X)) where X is an 8-bit unsigned number given input using Verilog HDL.

II. Generate GDS-II layout of design using Synopsys-flowscripts and report the following:
    1. Design area
    2. Clock frequency (for a positive setup slack value but less than 1)
    3. Power consumption

Github Repository Link: https://github.com/Huda147/log2_seq_huda

RTL2GDSII_PDEU

RTL Development

I. RTL Source Code

Filename: log2_seq.v

```
`timescale 1ns / 1ps

module log2_seq (
    input clk,
    input [7:0] X,
    output reg [2:0] Y
);

reg [2:0] Y_comb;

always @(*) begin
    if (X[7])       Y_comb = 3'd7;
    else if (X[6])  Y_comb = 3'd6;
    else if (X[5])  Y_comb = 3'd5;
    else if (X[4])  Y_comb = 3'd4;
    else if (X[3])  Y_comb = 3'd3;
    else if (X[2])  Y_comb = 3'd2;
    else if (X[1])  Y_comb = 3'd1;
    else if (X[0])  Y_comb = 3'd0;
    else            Y_comb = 3'd0;
end

always @(posedge clk) begin
    Y <= Y_comb;
end

endmodule
```

II. Testbench Code

Filename: log2_seq_tb.v

```
`timescale 1ns / 1ps

module log2_seq_tb

reg clk;
reg [7:0] X;
reg [7:0] X_d;
wire [2:0] Y;

log2_seq uut (
    .clk(clk),
    .X(X),
    .Y(Y)
);

always #5 clk = ~clk;

// Stimulus
initial begin
        $fsdbDumpfile("log2_seq.fsdb");
          $fsdbDumpvars();
end

always @(posedge clk) begin
    X_d <= X;
end

initial begin
    clk = 0;
    X = 0;

    #3 X = 8'd1;
    #10 X = 8'd2;
    #10 X = 8'd3;
    #10 X = 8'd7;
    #10 X = 8'd8;
    #10 X = 8'd15;
    #10 X = 8'd16;
    #10 X = 8'd28;
    #10 X = 8'd37;
    #10 X = 8'd99;
    #10 X = 8'd127;
    #10 X = 8'd128;
```

```
  #10 X = 8'd200;
  #10 X = 8'd255;
  #10 X = 8'd0;

  #20 $finish;
 end

always @(posedge clk) begin
  $display("Time=%0t  X_d=%0d  Y=%0d", $time, X_d, Y);
end

endmodule
```

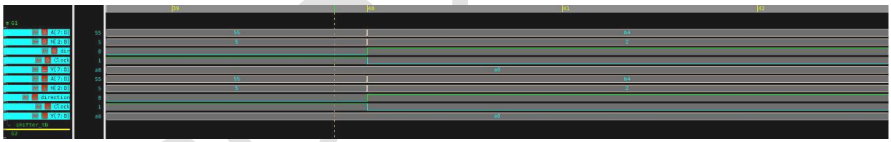
III. Testbench Simulation Commands

```
vcs -full64 log2_seq.v -debug_access+all -lca -kdb
vcs -full64 log2_seq_tb.v -debug_access+all -lca -kdb
./simv Verdi
verdi -ssf novas.fsdb -nologo
```

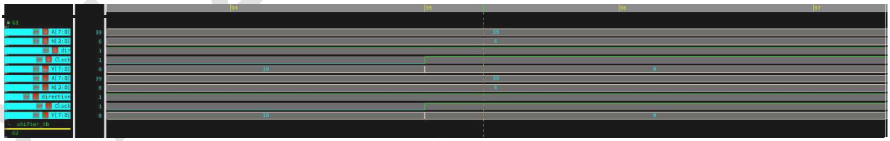
IV. Test Case Timing Diagrams

Note: The values of inputs and outputs are unsigned decimal number system.

1. X = 55 → Y = 5



2. X= 15 → Y= 3



Logic Synthesis

I. Changes in common_setup.tcl File

1. DESIGN_NAME : log2_seq
2. Target Library File : saed32rvt_ff1p16v125c.db

II. Constraint File

Filename: log2_seq.sdc

```
# Clock definition
 create_clock -period 10 -name "clk" [get_ports clk]
 set_clock_uncertainty 0.2 [get_clocks clk]
 set_clock_transition  0.1 [get_clocks clk]

# Input/Output delays
 set_input_delay  2.0 -clock clk [all_inputs]
 set_output_delay 2.0 -clock clk [all_outputs]

# False paths
# set_false_path -from [get_ports rst_n]
```

Parameters changed:
- Clock period
- Ports for max. input delay [To avoid warnings while reading constraint file]
- Ports for max. output delay [To avoid warnings while reading constraint file]
- Clock hold uncertainty [To adjust data required time]

RTL2GDSEU

III. Design Compiler TCL commands

Filename: run_dc.tcl

```
#================================
==============================
# Synopsys Design Compiler
#================================
==============================
# 1. SETUP & ENVIRONMENT
#-------------------------------------------
----------------

set DESIGN_NAME      "log2_seq"
set RTL_DIR          "../RTL/"set
```

RTL2GDSII_PD

```
set CONSTRAINTS_FILE "../constraints/${DESIGN_NAME}.sdc"
set LIBRARY_DIR      "../ref/lib/stdcell_rvt/"
set OUTPUT_DIR       "./outputs"
set REPORTS_DIR      "./reports"

# Target library (technology .db files)
set target_library   "$LIBRARY_DIR/saed32rvt_ss0p7vn40c.db"
set link_library     "* $LIBRARY_DIR/saed32rvt_ss0p7vn40c.db
$LIBRARY_DIR/saed32rvt_ff1p16v125c.db"

# Create output directories
file mkdir $OUTPUT_DIR
file mkdir $REPORTS_DIR

#-------------------------------------------------------------
# 2. READ RTL SOURCE FILES
#-------------------------------------------------------------

# Option A: Read Verilog
read_verilog [list \
    $RTL_DIR/${DESIGN_NAME}_rtl.v \
]

# Option B: Read SystemVerilog
# read_file -format sverilog [list \
#     $RTL_DIR/top_module.sv \
#     $RTL_DIR/submodule_a.sv \
# ]

# Option C: Read VHDL
# read_vhdl [list \
#     $RTL_DIR/top_module.vhd \
# ]

# Set the top-level design
current_design $DESIGN_NAME

#-------------------------------------------------------------
# 3. LINK THE DESIGN
#-------------------------------------------------------------

link
```

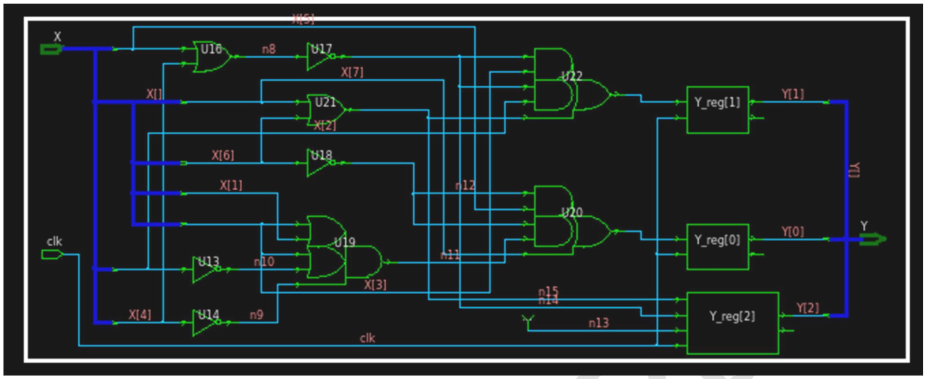
```
#-----------------------------------------------------------
# 4. SET DON'T USE CELLS
#-----------------------------------------------------------

#set_dont_use [get_lib_cells */FADD*]
#set_dont_use [get_lib_cells */HADD*]
#set_dont_use [get_lib_cells */AO*]
#set_dont_use [get_lib_cells */OA*]
#set_dont_use [get_lib_cells */NAND*]
#set_dont_use [get_lib_cells */XOR*]
set_dont_use [get_lib_cells */NOR*]
#set_dont_use [get_lib_cells */XNOR*]
#set_dont_use [get_lib_cells */MUX*]

#-----------------------------------------------------------
# 5. APPLY CONSTRAINTS (SDC)
#-----------------------------------------------------------

source $CONSTRAINTS_FILE

#-----------------------------------------------------------
# 6. COMPILE (SYNTHESIS)
#-----------------------------------------------------------

# Standard compile
# compile -map_effort medium -area_effort medium

# Ultra compile
  compile_ultra

# Incremental compile for further optimization (optional)
# compile_ultra -incremental

# Fix hold violations
#set_fix_hold [all_clocks]
#compile_ultra -incremental -only_hold_time

#-----------------------------------------------------------
# 7. POST-COMPILE OPTIMIZATIONS
#-----------------------------------------------------------

# Remove unconnected ports/nets
remove_unconnected_ports [get_cells -hierarchical *]

#-----------------------------------------------------------
# 8. REPORTS
#-----------------------------------------------------------

# Timing reports
report_timing -path full -delay max -max_paths 10 \
    > $REPORTS_DIR/timing_setup.rpt

report_timing -path full -delay min -max_paths 10 \
    > $REPORTS_DIR/timing_hold.rpt
```

```
# Area report
report_area -hierarchy \
   > $REPORTS_DIR/area.rpt

# Power report
report_power -hierarchy \
   > $REPORTS_DIR/power.rpt

# Design rule checks
report_constraint -all_violators \
   > $REPORTS_DIR/constraints.rpt

report_cell \
   > $REPORTS_DIR/cells.rpt

report_net \
   > $REPORTS_DIR/nets.rpt

# QoR summary
report_qor \
   > $REPORTS_DIR/qor.rpt

#-------------------------------------------------------------
# 9. WRITE OUTPUTS
#-------------------------------------------------------------

# Remove multi-port nets and constant logic assign statements from netlist
#set_fix_multiple_port_nets -all -buffer_constants

# Write gate-level netlist (Verilog)
                                 write -format verilog -hierarchy
   -output $OUTPUT_DIR/${DESIGN_NAME}_netlist.v

# Write SDC constraints
write_sdc $OUTPUT_DIR/${DESIGN_NAME}.sdc

#-------------------------------------------------------------
# 10. DONE
#-------------------------------------------------------------

echo "=========================================="
echo " Synthesis COMPLETE: $DESIGN_NAME"
echo "=========================================="
```

Parameters changed:

- RTL_SOURCE_FILES : “./../rtl_simulation/log2_seq.v”
- NOR cells set to don’t use [To increase no. of cells in design]
-

IV. Generated Outputs



1. Schematic diagram

2. Output File

Filename: log2_seq.mapped.v
Type: Verilog file

RTL2GDSII_P

Physical Design

1.Floorplanning

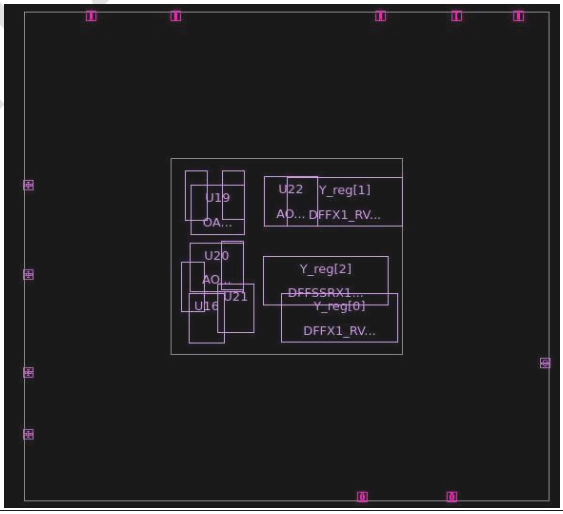
a. Floorplan

```
# scenario8 (UPDATED for full_adder)
open_block full_adder

initialize_floorplan \
    -core_utilization 0.6 \
    -core_offset {5 5}

set_individual_pin_constraints \
    -ports [get_ports {clk X[*]}] \
    -sides {1 2} \
    -pin_spacing_distance 2

set_individual_pin_constraints \
    -ports [get_ports {Y[*]}] \
    -sides {3 4} \
    -pin_spacing_distance 2

place_pins -self
create_placement -floorplan -effort medium
```

Parameters changed:

- Library name : LOG2_SEQ_LIB
- Initial block name : LOG2
- Set pin constraints and removed blockage in scenario (scenario 8 used)
- Read constraints file [To check timing report]

b. Design Layout



2. Powerplanning

a. Powerplan

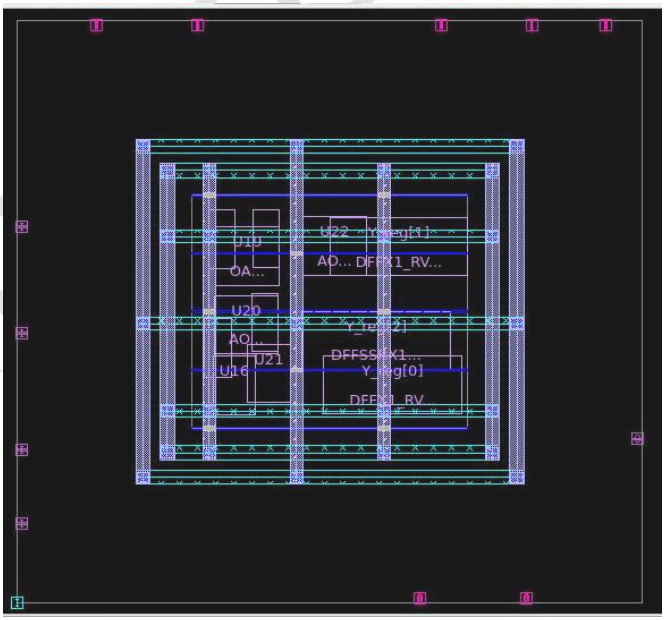
```
#==============================================================
# Synopsys IC Compiler II - Script 04: Power Planning
# File: 04_power_planning.tcl
#==============================================================

source scripts/01_setup.tcl

#--------------------------------------------------------------
# 4a. Create and connect power / ground nets
#--------------------------------------------------------------

create_net -power {VDD}
create_net -ground {VSS}

connect_pg_net -all_blocks -automatic

#--------------------------------------------------------------
# 4b. Power / ground rings
#--------------------------------------------------------------

create_pg_ring_pattern core_ring_pattern -horizontal_layer M7 -
horizontal_width .4 -horizontal_spacing .3 -vertical_layer M8 -vertical_width .4
-vertical_spacing .3

set_pg_strategy core_power_ring -core -pattern {{name :
core_ring_pattern}{nets : {VDD VSS}}{offset : {.5 .5}}}

compile_pg -strategies core_power_ring

#--------------------------------------------------------------
# 4c. Power mesh
#--------------------------------------------------------------

create_pg_mesh_pattern mesh -layers { {{vertical_layer: M6}{width: .34}
{spacing: interleaving}{pitch: 5} {offset: .5}} {{horizontal_layer: M7}{width: .38}
{spacing: interleaving} {pitch: 5} {offset: .5}} {{vertical_layer: M8}{width: .38}
{spacing: interleaving}{pitch: 5} {offset: .5}}}

set_pg_strategy core_mesh -pattern { {pattern:mesh} {nets: VDD VSS}} -core -
extension {stop: innermost_ring}

compile_pg -strategies core_mesh
```

```
# 4d. Standard-cell power rails
#-------------------------------------------------------------

create_pg_std_cell_conn_pattern std_cell_rail -layers {M1} -rail_width 0.06

set_pg_strategy rail_strat -core -pattern {{name: std_cell_rail} {nets: VDD VSS} }

compile_pg -strategies rail_strat

#-------------------------------------------------------------
# 4e. Connectivity check
#-------------------------------------------------------------

check_pg_connectivity \
   > $REPORTS_DIR/pg_connectivity.rpt

save_block
save_lib

puts "INFO: Power planning complete."
```

Parameters changed:

- PG ring pattern : horizontal and vertical width and spacing
- PG mesh pattern : Width, pitch and offset of all three layers
- PG mesh strategy : Added directions right s bottom

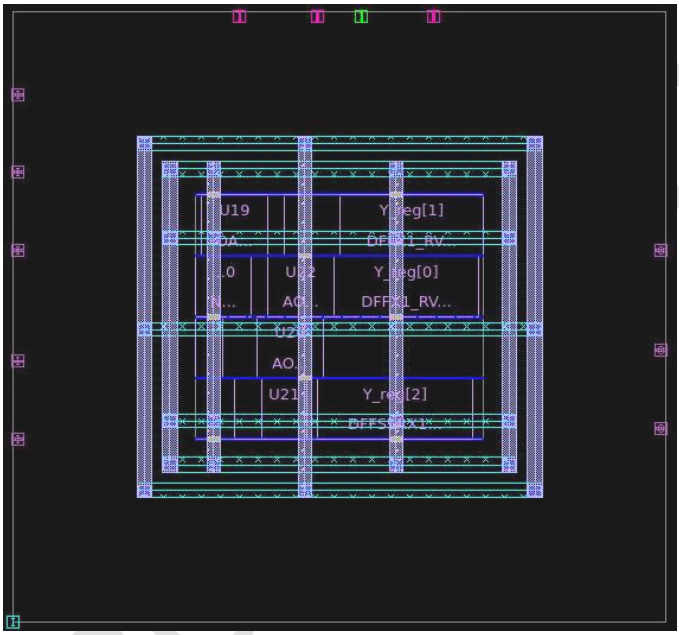
b. Design Layout



3. Placement

Parameters changed:
-NOR cell to don't use [Similar constraint during Logic Synthesis]

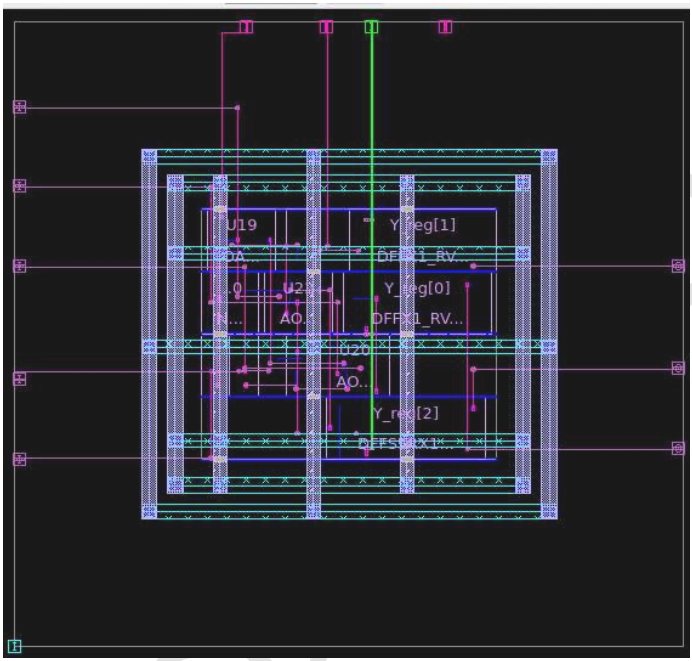
a. Design layout



RTL2GDSII

4. Clock Tree Synthesis

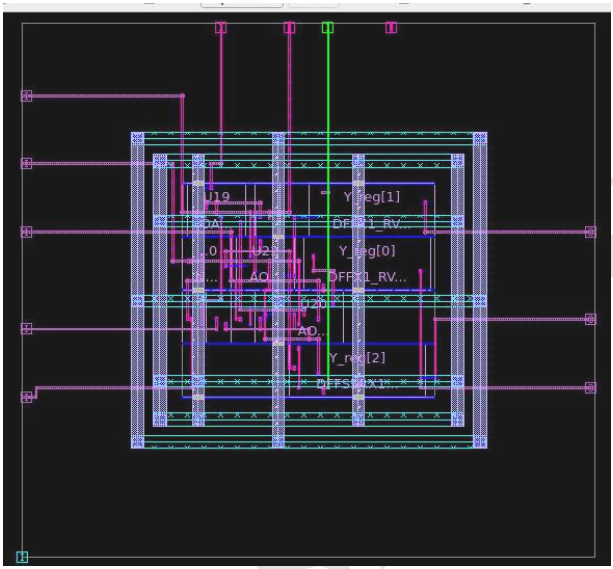
a. Design layout



RTL2GDSII

5. Routing

a. Design layout



b. Report contents

Slack:	0 ns
Power consumption:	1.053 W
Area:	41.63

c. Output Files

a. **Constraint File:** log2_seq.sdc
b. **Verilog File:** log2_seq_routed.v
c. **SPEF File:** log2_func::nom.spef.p1_125.spef
d. **SPEF File:** log2_func::nom.spef.p2_125.spef
   i. log2_func::nom.spef.spef_scenario (Provides the operating conditions for the .spef files)

RTL2GDS-EU

Static Timing Analysis

I. Late Corner

1. TCL commands

```
set report_default_significant_digits 6
set link_path "./../ref/lib/stdcell_rvt/saed32rvt_ff1p16v125c.db"

read_verilog "./../ICCII/results/.routed.v" link_design
current_design

read_sdc "./../CONSTRAINTS/log2_seq.sdc"

read_parasitics "./../ICCII/results/log2_func::nom.spef.p1_125.spef"

update_timing -full

report_timing > ./reports/timing/timing.p1_report report_design
> ./reports/design/design.p1_report

check_timing -verbose > ./reports/check_timing/check_timing.p1_report
report_global_timing > ./reports/timing/report_global_timing.p1_report report_clock -skew -
attribute > ./reports/clock/report_clock.p1_report report_analysis_coverage >
./reports/analysis_coverage/report_analysis_coverage.p1_report
report_timing -slack_lesser_than 0.0 -delay_type min_max -nosplit -input -net >
./reports/timing/report_timing.p1_report
```

Parameters changed:

- Library : saed32rvt_ff1p16v125c.db
- Verilog File read : log2_seq.routed.v
- Constraints File read : log2_seq.sdc
- SPEF File read : log2_func::nom.spef.p1_125.spef

2. Report contents

Slack:	0 ns No setup or hold violations

Learnings

I. RTL Development

- Designed and implemented digital circuits using **behavioral modeling in Verilog HDL**, focusing on writing synthesizable and modular code.
- Developed structured Verilog modules using constructs such as always blocks, conditional statements, and case-based logic for efficient hardware description.
- Created comprehensive **testbenches** to validate functionality by applying different input stimulus and observing output responses.
- Performed simulation using **Synopsys VCS**, ensuring functional correctness of the design.
- Utilized **Synopsys Verdi** for waveform analysis, signal tracing, and debugging, enabling efficient identification of logical errors and mismatches.

II. Logic Synthesis

- Converted RTL designs into gate-level netlists using **Synopsys Design Compiler**, targeting standard cell libraries.
- Applied timing, area, and power constraints using SDC (Synopsys Design Constraints) to guide synthesis optimization.
- Iteratively refined constraints such as **clock definitions, input/output delays, and load capacitances** to meet design specifications.
- Analyzed synthesis reports including **area reports, timing reports, and power reports** to evaluate design efficiency.
- Studied critical timing parameters such as **Worst Negative Slack (WNS)** and **Total Negative Slack (TNS)** to identify violations.
- Gained insights into **process corners (PVT variations)**—process, voltage, and temperature—and their effect on circuit performance and timing margins.

III. Physical Design

- Performed full physical design flow using **Synopsys IC Compiler II**, transforming the synthesized netlist into a physical layout.
- Created an optimized **floorplan**, defining core area, aspect ratio, placement of macros, and IO pin configuration.
- Developed a robust **power plan** (power rings, straps, and grids) to ensure reliable power distribution and minimize IR drop.
- Executed **placement of standard cells**, optimizing for congestion, timing, and power efficiency.
- Verified design integrity using connectivity checks to ensure proper net connections and no floating nodes.
- Performed **Clock Tree Synthesis (CTS)** to distribute clock signals with minimal skew and controlled latency.
- Analyzed clock parameters such as **insertion delay, skew, and jitter** to ensure timing reliability.
- Carried out **routing (global and detailed routing)** to establish all physical interconnections while minimizing congestion and crosstalk.
- Conducted post-routing checks including **Design Rule Checks (DRC)** and layout verification to ensure manufacturability.

RTL2GDSII_PDEU

IV. Static Timing Analysis (STA)

- Performed timing verification using **Synopsys PrimeTime** to ensure the design meets all timing constraints.
- Analyzed **setup and hold timing paths**, identifying critical paths and timing bottlenecks.
- Interpreted detailed STA reports including slack analysis, path delays, and violation summaries.
- Evaluated timing across multiple **process corners and operating conditions** for robust design validation.
- Applied timing fixes such as **buffer insertion, gate resizing, and path optimization** to resolve violations.
- Ensured timing closure and verified that the design meets required performance specifications before final sign-off.

RTL2GDSII_PDEU